

.NET MICROSERVICES

20 Experienced-Level Interview Questions and Answers

Concepts • Trade-offs • Real Project Examples • C# Code • Azure Architecture

Designed for: Senior .NET Developers, Technical Leads and Solution Architects

Includes interview-ready explanations covering synchronous and asynchronous communication, DDD, CQRS, Saga, messaging, resiliency, security, databases, Docker, Kubernetes/AKS, observability and complete architecture.

Interview strategy

For every answer, explain the business need, the design decision, the failure scenario and the trade-off. Senior interviews evaluate judgment—not only definitions.

Table of Contents

1. How do Microservices communicate? Synchronous vs asynchronous communication
2. What are Microservices? Microservices vs Monolithic architecture
3. How do you identify service boundaries and break a Monolith?
4. What is CQRS? Why is it useful and when should you not use it?
5. How do you handle failures: Retry, Circuit Breaker, Timeout and fallback?
6. How do you secure service-to-service communication?
7. What is DDD and how does it help design Microservices?
8. What is the Saga Pattern? How do distributed transactions work?
9. How do you design Event-Driven Microservices using Azure Service Bus or Kafka?
10. How do you handle duplicate messages and implement idempotency?
11. What is an API Gateway and what role does it play?
12. How do you deploy .NET Microservices using Docker?
13. How do you deploy and scale .NET Microservices on Kubernetes/AKS?
14. How do you design the database layer? Should every service own a database?
15. What is Eventual Consistency and how do you maintain consistency?
16. What is Event Sourcing? How is it different from CQRS?
17. How do you handle message failures, retries and Dead-Letter Queues?
18. How do you implement Rate Limiting in Microservices?
19. How do you make Microservices scalable, highly available and fault tolerant?
20. Draw and explain a complete real-world .NET Microservices architecture

1. How do Microservices communicate? Synchronous vs asynchronous communication

Microservices communicate either synchronously, where the caller waits for an immediate response, or asynchronously, where a producer publishes a message and continues. A mature design uses both and selects the method according to business consistency, latency and coupling requirements.

Synchronous communication

- REST/HTTP is simple and interoperable; gRPC is efficient for internal strongly typed communication.
- Use it when the caller requires an immediate query result or confirmation.
- The caller and receiver are runtime-coupled. Long call chains increase latency and the probability of cascading failure.

Asynchronous communication

- Use Azure Service Bus, Kafka or RabbitMQ for events, commands and background processing.
- The producer does not wait for all consumers, so services are temporally decoupled and traffic spikes can be buffered.
- It introduces eventual consistency, duplicate delivery, ordering and operational monitoring concerns.

Example flow

```
Client -> Order API -> Inventory API      (synchronous query)
Order Service -> OrderCreated topic      (asynchronous event)
                    | -> Payment Service
                    | -> Notification Service
                    ` -> Analytics Service
```

Typed HTTP client

```
builder.Services.AddHttpClient<IInventoryClient, InventoryClient>(client =>
{
    client.BaseAddress = new Uri(config["Services:Inventory"]);
    client.Timeout = TimeSpan.FromSeconds(3);
});

public async Task<bool> IsAvailableAsync(Guid productId, int quantity,
    CancellationToken ct)
{
    using var response = await http.GetAsync(
        $"/api/inventory/{productId}?quantity={quantity}", ct);
    response.EnsureSuccessStatusCode();
    return (await response.Content.ReadFromJsonAsync<Availability>(ct)).Available;
}
```

Experienced interview answer: I use synchronous calls for immediate results and asynchronous events for state propagation and long-running workflows. I avoid Gateway → Order → Customer → Inventory → Pricing chains. Every remote call has a timeout and cancellation token; event consumers are idempotent.

Area	Monolith	Microservices
Deployment	Whole application	Independent service
Scaling	Scale complete application	Scale selected service
Transactions	Local ACID is easier	Saga/eventual consistency
Communication	In-process	Network and messaging
Data	Usually shared	Owned per service

Area	Monolith	Microservices
Operations	Simpler	Requires mature DevOps/observability

2. What are Microservices? Microservices vs Monolithic architecture

A microservice is an independently deployable component aligned to a business capability. It owns its domain logic and data and exposes explicit API or event contracts. It is not merely a small controller project.

Advantages

- Independent deployments and scaling
- Fault isolation and smaller codebases
- Team autonomy and different release cadence

Trade-offs

- Network latency and partial failures
- Distributed transactions and eventual consistency
- Contract versioning, tracing, testing and infrastructure cost

Decision guidance

A modular monolith is often better for a small team, unclear domain boundaries or a product at an early stage. Microservices create value only when independent ownership, release and scaling justify the operational cost.

Experienced interview answer: Microservices are an organizational and business-boundary decision, not just a technical decomposition. I would not split an application only because it is large.

3. How do you identify service boundaries and break a Monolith?

Service boundaries must follow business capabilities and bounded contexts—not technical layers or individual tables. For payroll, good candidates are Attendance, Leave, Payroll Calculation and Payslip rather than ControllerService or EmployeeTableService.

Discovery process

- Map business capabilities, workflows, rules and vocabulary with domain experts.
- Find data and rules that change together; define one owner.
- Check whether the capability can be deployed, scaled and operated independently.
- Use the Strangler Fig Pattern to route selected functionality to new services while the monolith continues running.

Practical extraction sequence

Client -> Gateway -> New Leave Service
 -> New Payroll Service
 -> Remaining Monolith

Phase 1: modularize inside the monolith
Phase 2: extract one low-risk bounded context
Phase 3: replace shared-table access with APIs/events

Experienced interview answer: I first make domain boundaries visible in a modular monolith, then extract one high-value, clearly owned capability. Each extracted service must own its rules and data;

otherwise we create a distributed monolith.

4. What is CQRS? Why is it useful and when should you not use it?

CQRS separates operations that change state (commands) from operations that return data (queries). It may be implemented with separate handlers over one database or with independently optimized read and write stores.

Command and query examples

```
public record CreateOrderCommand(Guid CustomerId,
    IReadOnlyCollection<OrderItemDto> Items);
public record GetOrderQuery(Guid OrderId);

// Intent-oriented commands: ApproveOrder, CancelOrder
// Avoid table-oriented commands: UpdateOrderTable
```

Benefits

- Different read and write models
- Independent performance and security optimization
- Clear business intent and support for denormalized projections

When not to use it

- Simple CRUD where both models are identical
- Low scale with no reporting or model complexity
- When eventual consistency and extra infrastructure add no business value

Experienced interview answer: I treat CQRS as a spectrum. I often begin with separate command/query handlers and one database, introducing separate stores only when scaling or model complexity proves the need. CQRS does not require Event Sourcing.

5. How do you handle failures: Retry, Circuit Breaker, Timeout and fallback?

Distributed systems must expect slow responses, transient faults and unavailable dependencies. Resilience policies need strict time budgets and must be safe for the operation being executed.

Patterns

- Timeout: stop waiting beyond the permitted latency.
- Retry: repeat only transient and safe operations.
- Exponential backoff with jitter: spread retries to avoid a retry storm.
- Circuit breaker: fail fast after repeated dependency failures; probe recovery in half-open state.
- Fallback: return cached or reduced data only when it remains truthful and safe.

Modern .NET setup

```
builder.Services
    .AddHttpClient<IInventoryClient, InventoryClient>()
    .AddStandardResilienceHandler();

// Configure per-attempt timeout, overall timeout, retry count,
// transient status codes and circuit-breaker thresholds per dependency.
```

Important warning

Never blindly retry a payment POST. The first request may have succeeded even when its response was lost. Use an idempotency key and store the result.

Experienced interview answer: A retry is not a universal fix. I calculate the complete retry duration against the incoming request deadline, use jitter, and protect non-idempotent commands with a business idempotency key.

6. How do you secure service-to-service communication?

Service-to-service security covers identity, authorization, transport, secrets and network access. Internal network location alone must not be treated as trust.

Recommended controls

- OAuth2 client-credentials flow for application identities
- Managed Identity/Workload Identity for Azure resources
- JWT validation: signature, issuer, audience, expiry and scopes/roles
- TLS everywhere; mTLS for higher-trust environments
- Key Vault, private endpoints, network policies and least privilege

Token flow

Order Service -> Identity Provider: client credentials
 Identity Provider -> Order Service: short-lived access token
 Order Service -> Payment Service: Bearer token
 Payment Service: validate audience + scope + caller role

Gateway responsibility

APIM can validate external tokens, apply WAF and rate limits, but each downstream service still authorizes its own operations. Forward user tokens only when downstream user context is genuinely required.

Experienced interview answer: I apply zero-trust principles. On Azure I prefer Managed Identity for platform resources and OAuth2 client credentials for APIs; authorization is enforced by the owning service, not only at the gateway.

7. What is DDD and how does it help design Microservices?

Domain-Driven Design aligns the software model with business language and rules. Strategic DDD discovers boundaries; tactical DDD models complex behavior inside a boundary.

Core terminology

- Domain and subdomain: overall business and its specialized parts
- Bounded Context: boundary within which a model has one meaning
- Ubiquitous Language: terms shared by experts and developers
- Entity: object with identity and lifecycle
- Value Object: immutable concept defined by its values
- Aggregate Root: entry point enforcing consistency rules

Aggregate example

```
Order (Aggregate Root)
| - OrderItem
| - ShippingAddress (Value Object)
` - PaymentStatus

// External code must not edit OrderItem directly.
order.AddItem(productId, quantity, price);
```

Experienced interview answer: A bounded context is a strong service candidate, but every entity is not a microservice. I use rich aggregates for complex rules and simpler CRUD models for reference data.

8. What is the Saga Pattern? How do distributed transactions work?

A Saga coordinates a business transaction across service-owned databases through local transactions and compensating actions. Compensation is a new business operation—not a global database rollback.

Order Saga

```
Create Pending Order
-> Reserve Inventory
-> Charge Payment
-> Confirm Order
```

```
If payment fails:
-> Release Inventory
-> Mark Order PaymentFailed
```

Choreography vs orchestration

- Choreography: services react to events; lightweight but workflows become difficult to see as they grow.
- Orchestration: a Saga component sends commands and records state; clearer for complex, auditable workflows but must avoid becoming a god service.

Operational requirements

- Persist SagaId, business ID, current step, status, timeout and last error.
- Make every action and compensation idempotent.
- Support retries, manual intervention and reconciliation for unresolved cases.

Experienced interview answer: For payments, inventory and shipment I generally prefer orchestration because state, timeout and compensation are easier to audit. A refund is not a technical rollback; it is a separate, traceable business transaction.

9. How do you design Event-Driven Microservices using Azure Service Bus or Kafka?

Event-driven architecture communicates business facts through a broker. Domain events stay inside a bounded context; stable integration events cross service boundaries.

Azure Service Bus vs Kafka

- Service Bus: enterprise queues/topics, commands, DLQ, scheduled delivery, sessions and peek-lock.
- Kafka: high-throughput event streaming, durable log, replay and independent consumer groups.

Reliable publication

```
Business transaction:
INSERT Order
INSERT OutboxMessage(OrderCreatedV1)
COMMIT
```

```
Background publisher:
read unsent outbox -> publish -> mark sent
```

Message envelope

- EventId and event type/version
- OccurredAtUtc, CorrelationId and CausationId
- Business entity ID and minimal stable payload

Experienced interview answer: I use the Transactional Outbox Pattern so the state change and outgoing event commit atomically in one database. Consumers assume at-least-once delivery and are idempotent.

10. How do you handle duplicate messages and implement idempotency?

Duplicate delivery is normal in reliable messaging. Idempotency means processing the same logical request or message more than once has the same effect as processing it once.

Inbox transaction

```
BEGIN TRANSACTION
  IF ProcessedMessages contains (MessageId, ConsumerName)
    COMMIT and acknowledge
  ELSE
    apply business update
    insert ProcessedMessages row
  COMMIT
Acknowledge message only after commit
```

Database failure after consumption

- Use peek-lock/visibility timeout rather than destructive receive.
- Do not acknowledge if the database transaction fails.
- On redelivery, the stored message ID and unique constraints prevent double processing.

HTTP idempotency

For payment creation, accept an Idempotency-Key, store it with the original outcome and return that outcome on a retry.

Experienced interview answer: Broker duplicate detection is only supplementary and may have a time window. Consumer-side idempotency plus database uniqueness is the final protection.

11. What is an API Gateway and what role does it play?

An API Gateway is the controlled external entry point for clients. Common .NET/Azure choices include Azure API Management, YARP and Kubernetes Ingress.

Responsibilities

- Routing and backend hiding
- Authentication, TLS, WAF and rate limiting
- Request transformation, versioning and caching
- Correlation IDs, access logs and limited response aggregation

APIM and YARP

APIM provides managed policies, subscriptions, developer portal, quotas and analytics. YARP is a customizable .NET reverse proxy. They may be combined with APIM at the public edge and YARP/BFF for application-specific routing.

Boundary

Do not place order, payroll or payment business rules in the gateway. Domain authorization remains in the owning service.

Experienced interview answer: I use the gateway for edge and cross-cutting concerns. Excessive aggregation can recreate a distributed monolith and become a latency bottleneck.

12. How do you deploy .NET Microservices using Docker?

Docker packages a .NET service and its runtime dependencies into an immutable image. A multi-stage Dockerfile uses the SDK to build and a smaller ASP.NET runtime image to execute.

Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY ["Order.Api/Order.Api.csproj", "Order.Api/"]
RUN dotnet restore "Order.Api/Order.Api.csproj"
COPY . .
RUN dotnet publish "Order.Api/Order.Api.csproj" -c Release \
    -o /app/publish --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
USER $APP_UID
EXPOSE 8080
ENTRYPOINT ["dotnet", "Order.Api.dll"]
```

Production practices

- Run as non-root and never bake secrets into images.
- Write logs to stdout/stderr; keep the container stateless.
- Pin supported base versions, scan images and use .dockerignore.
- Build once and promote the same image through QA, staging and production.

Experienced interview answer: My pipeline tests, builds, scans and pushes one versioned image to the registry. Deployment injects environment configuration and secrets; it does not rebuild the image.

13. How do you deploy and scale .NET Microservices on Kubernetes/AKS?

Kubernetes runs containerized services and maintains their desired state. AKS is Azure's managed Kubernetes offering.

Core components

- Pod runs containers; Deployment manages replicas and rolling updates.
- Service supplies a stable internal endpoint; Ingress routes HTTP traffic.
- ConfigMap stores non-sensitive configuration; Key Vault/Secrets Store supplies secrets.
- HPA scales pods; cluster autoscaler scales nodes; node pools isolate workload types.

Probe guidance

```
readinessProbe:
  httpGet: { path: /health/ready, port: 8080 }
livenessProbe:
  httpGet: { path: /health/live, port: 8080 }
resources:
  requests: { cpu: "200m", memory: "256Mi" }
  limits: { cpu: "1", memory: "512Mi" }
```

Availability

- Run multiple replicas across nodes/zones.
- Use Pod Disruption Budgets, rolling updates and workload identity.
- For message consumers, scaling by queue depth is often more useful than CPU.

Experienced interview answer: I keep liveness independent of external databases; otherwise a database outage can restart every pod. Readiness decides traffic eligibility, while liveness detects a stuck process.

14. How do you design the database layer? Should every service own a database?

Database per service means data ownership and access control. It does not always require a separate physical database server for every service.

Isolation options

- Separate physical databases
- Separate logical databases on one server
- Separate schemas with strict permissions as a migration stage

Why shared databases cause trouble

- Cross-service joins create runtime and schema coupling.
- Any service can bypass another service's business rules.
- Deployments and scaling become coordinated.

Cross-service reads

- Call an API for immediate authoritative data.
- Subscribe to events and maintain a local read model.
- Use API composition for limited queries.
- Use a warehouse/lake/CDC pipeline for reporting.

Experienced interview answer: No other service directly updates or queries the owner's tables. In migration, separate schemas can be an intermediate step, but ownership must already be enforced.

15. What is Eventual Consistency and how do you maintain consistency?

Eventual consistency means service-owned views may temporarily differ but converge after messages are processed. It is a normal outcome of autonomous services rather than an unhandled error.

Consistency toolkit

- Transactional Outbox and idempotent Inbox
- Saga and compensating transactions
- Ordered delivery or entity sequence numbers where needed
- Optimistic concurrency, reconciliation jobs and message-lag monitoring

User-visible states

Order: Pending
Payment: Processing
Inventory: ReservationPending

Later -> Confirmed OR PaymentFailed

Boundary rule

Keep strong ACID consistency inside one aggregate/service transaction. If a rule demands instant atomic updates across multiple services, reconsider the boundary or use reservation semantics.

Experienced interview answer: I expose meaningful "processing" states instead of pretending all services update instantly. We measure lag and reconcile stuck workflows.

16. What is Event Sourcing? How is it different from CQRS?

Event Sourcing persists an append-only sequence of domain events rather than only the latest state. Current state is rebuilt by replaying events, often with snapshots for performance.

Example stream

```
OrderCreated  
ItemAdded  
PaymentCompleted  
OrderConfirmed  
OrderShipped
```

Difference from CQRS

CQRS separates command and query models. Event Sourcing is a persistence strategy. Either can be used without the other, although they are frequently combined.

When it fits

- Financial ledgers and audit-sensitive workflows
- Complex lifecycle reconstruction and temporal queries
- Domains where the history of how state changed has business value

Challenges

- Event versioning, replay, snapshots and eventual projections
- Privacy/deletion obligations and operational complexity
- External side effects must never execute during replay

Experienced interview answer: I avoid Event Sourcing for ordinary CRUD. Its audit and temporal benefits must clearly justify the long-term schema and operational complexity.

17. How do you handle message failures, retries and Dead-Letter Queues?

Message handling must distinguish transient failures from poison messages. Retries help temporary outages; repeated invalid messages belong in a Dead-Letter Queue.

Failure policy

- Retry transient database/network/throttling faults with bounded backoff and jitter.
- Do not repeatedly retry invalid schema or permanent business validation failures.
- After maximum deliveries, dead-letter with reason, description and diagnostic context.

DLQ operating model

- Alert on count and age, not merely existence.
- Inspect payload, event version, correlation ID and stack trace.
- Fix the cause, then replay through a controlled and audited process.
- Quarantine unrecoverable messages.

Ordering

Use Service Bus sessions or Kafka partition keys for per-entity order. Include entity version numbers so consumers can detect stale or missing events.

Experienced interview answer: A DLQ is an operational safety mechanism, not the final solution. Ownership, alerts, diagnosis, replay authorization and reconciliation must be defined before production.

18. How do you implement Rate Limiting in Microservices?

Rate limiting protects capacity and ensures fair usage. Policies can be partitioned by user, client application, tenant, API key or endpoint.

ASP.NET Core example

```
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("client-policy", limiter =>
    {
        limiter.PermitLimit = 100;
        limiter.Window = TimeSpan.FromMinutes(1);
        limiter.QueueLimit = 0;
    });
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
});
app.UseRateLimiter();
app.MapControllers().RequireRateLimiting("client-policy");
```

Deployment levels

- Gateway quota protects the platform before traffic reaches services.
- Service concurrency limiter protects expensive local operations.
- In-memory counters are per replica; global quotas require APIM, Redis or another distributed limiter.

Response

Return HTTP 429 and, where practical, Retry-After. Do not use IP as the only identity because NAT groups users and attackers can rotate addresses.

Experienced interview answer: I combine gateway client quotas with service-level concurrency protection. The partition key comes from authenticated tenant/client identity.

19. How do you make Microservices scalable, highly available and fault tolerant?

Scalability handles growth, high availability keeps the service accessible and fault tolerance preserves or safely degrades behavior during failures.

Architecture measures

- Stateless services and horizontal replicas behind load balancing
- Queues for buffering and independent consumer scaling
- Redis caching with expiration, invalidation and stampede protection
- Timeout, retry, circuit breaker, bulkhead and idempotency
- Zone redundancy, managed failover, PDBs and rolling deployment

Observability

- Structured logs, metrics and distributed traces
- Correlation/trace IDs across HTTP and messages
- P50/P95/P99 latency, error rate, saturation, queue depth and consumer lag
- SLOs and alerts tied to user impact

Health checks

```
builder.Services.AddHealthChecks()
    .AddSqlServer(connectionString, tags: ["ready"])
    .AddRedis(redisConnection, tags: ["ready"]);

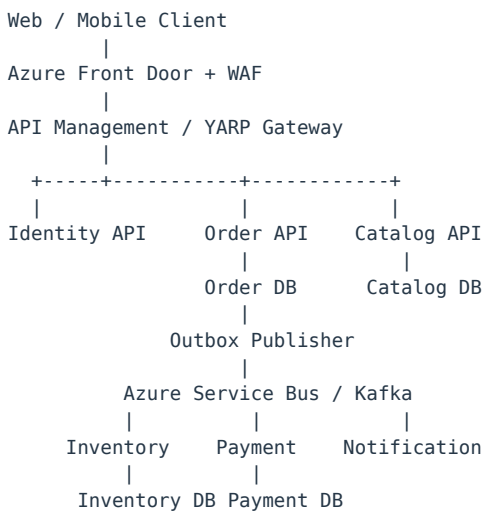
// /health/live: process is alive
// /health/ready: instance can accept traffic
```

Experienced interview answer: Adding API pods does not help when SQL or a downstream service is the bottleneck. I load-test, trace dependencies and scale or redesign the actual constrained resource.

20. Draw and explain a complete real-world .NET Microservices architecture

A complete Azure-based design separates edge protection, independently deployable services, service-owned data, event processing and observability.

Reference architecture



Shared platform: Redis | Key Vault | App Insights/OpenTelemetry | AKS

End-to-end order flow

- Client authenticates and sends a token through APIM.
- Order Service validates, creates Pending order and Outbox event in one transaction.
- Publisher sends OrderCreated; a Saga reserves inventory and processes payment.
- Failure triggers idempotent compensation; unresolved cases are reconciled.
- Every service runs as a versioned container with multiple replicas and independent CI/CD.

Cross-cutting design

- Managed Identity/OAuth2, least privilege and service-level authorization
- Timeouts, circuit breakers, retries with jitter and idempotency
- Database per service, read projections and eventual consistency
- OpenTelemetry trace propagation through APIs and messages
- Dashboards for latency, errors, saturation, queue lag, DLQ and Saga failures

Experienced interview answer: In my architecture, synchronous communication is limited to immediate needs; Service Bus handles state propagation and workflows. Saga, Outbox and idempotent Inbox replace cross-database transactions. Stateless .NET services run on AKS and scale independently, while APIM, Managed Identity, Redis and Application Insights provide edge control, security, performance and observability.

Senior Interview Closing Framework

For every microservices question, connect the answer to these five dimensions:

Dimension	What to explain
1. Business boundary	Capability, bounded context and ownership
2. Communication	REST/gRPC vs events and contract design
3. Consistency	Local transaction, Outbox, Saga and idempotency
4. Reliability/security	Timeouts, retries, circuit breakers, OAuth2 and least privilege
5. Operations	Deployment, scaling, tracing, health checks and recovery

Strong closing statement: Microservices give independent ownership, deployment and scaling, but they exchange in-process simplicity for network, consistency and operational complexity. My design decisions make those trade-offs explicit and measurable.